

SMART LEARNING RECOMMENDATION PLATFORM

Name: Elamathy R K

Project Type: AI/ML Based Web Application

Technology Stack: React.js, Flask, Python, Machine Learning

Project Status: Completed

INTRODUCTION

Online learning platforms have become an essential part of modern education and skill development. However, most traditional platforms provide the same course suggestions to every user without considering individual interests, learning goals, or preferences. This creates difficulty for learners in identifying suitable courses from large collections of educational content.

The Smart Learning Recommendation Platform is an AI-powered web application developed to provide personalized course recommendations based on user interests and learning preferences. The system combines machine learning techniques with a responsive web interface to create an intelligent learning environment that improves user engagement and course discovery.

The platform allows users to create accounts, select preferred learning domains, explore trending courses, search and filter educational content, save courses for later learning, and receive AI-based personalized recommendations dynamically. The recommendation system uses Sentence Transformers and cosine similarity to analyze semantic similarity between user interests and available course content.

The project demonstrates the practical implementation of full-stack web development integrated with machine learning-based recommendation systems using React.js and Flask architecture.

OBJECTIVES

- To develop an intelligent course recommendation platform
- To implement AI-based personalized learning suggestions
- To provide an interactive and responsive user interface
- To integrate frontend and backend systems using REST APIs
- To build a machine learning-based recommendation engine
- To implement search, filtering, and category-based course exploration
- To provide trending and highly rated course recommendations
- To implement saved learning collections for users
- To demonstrate practical application of content-based filtering techniques

TECHNOLOGIES USED

Frontend Technologies

1. React.js

Used for developing the frontend user interface using component-based architecture.

2. React Router DOM

Used for navigation and routing between application pages.

3. Axios

Used for API communication between React frontend and Flask backend.

4. CSS / Inline Styling

Used for responsive layouts, UI styling, spacing, gradients, and visual enhancements.

5. React Icons

Used for adding modern icons and improving dashboard visualization.

Backend Technologies

1. Python

Used for backend development and machine learning implementation.

2. Flask

Used for creating backend APIs and handling server-side operations.

3. Flask-CORS

Used for enabling communication between frontend and backend running on different ports.

4. REST APIs

Used for frontend-backend data exchange and recommendation requests.

Machine Learning Technologies

1. Sentence Transformers

Used for generating semantic embeddings from course information and user interests.

2. all-MiniLM-L6-v2 Model

Used for converting textual data into vector embeddings for recommendation processing.

3. Cosine Similarity

Used for measuring similarity between user interests and available courses.

4. Pandas

Used for dataset loading, preprocessing, filtering, and manipulation.

5. Content-Based Filtering

Used for generating personalized recommendations based on course features.

Dataset & Storage

1. CSV Dataset

Used for storing course information such as titles, categories, ratings, levels, and tags.

2. Browser LocalStorage

Used for storing user login status, preferences, and saved courses temporarily.

PROJECT ARCHITECTURE

The Smart Learning Recommendation Platform follows a client-server architecture where the React frontend communicates with the Flask backend using REST APIs.

The frontend is responsible for user interaction, UI rendering, navigation, filtering, and displaying recommendation results. The backend processes recommendation requests, performs machine learning computations, filters courses, and returns personalized results to the frontend.

The recommendation engine uses machine learning embeddings and cosine similarity to identify semantically relevant courses for users.

SYSTEM WORKFLOW

Step 1 User Registration/Login

Users create accounts or log into the platform using authentication pages.

Step 2 Preference Selection

Users select their preferred learning domains and interests.

Step 3 Frontend API Request

The frontend sends user preferences to the Flask backend through REST APIs.

Step 4 Dataset Processing

The backend loads and processes course data from the CSV dataset.

Step 5 Embedding Generation

Sentence Transformers generate embeddings for courses and user interests.

Step 6 Similarity Matching

Cosine similarity is calculated between user preference embeddings and course embeddings.

Step 7 Recommendation Ranking

Courses are ranked based on similarity scores and ratings.

Step 8 Recommendation Display

The frontend displays personalized AI-based course recommendations.

Step 9 User Interaction

Users can explore, save, search, filter, or launch recommended courses.

RECOMMENDATION MODEL USED

Sentence Transformer Model

Model Name

all-MiniLM-L6-v2

Library

SentenceTransformers

Purpose

The model converts textual course information and user interests into numerical vector embeddings. These embeddings help the system understand semantic similarity between courses and user preferences.

Recommendation Technique

The platform uses:

- Content-Based Filtering
- Embedding-Based Similarity Matching
- Cosine Similarity Scoring

Recommendation Logic

The recommendation engine performs the following operations:

- Combines course title, category, and tags
- Generates embeddings for all courses
- Generates embeddings for user-selected interests
- Computes cosine similarity scores
- Boosts recommendation scores using course ratings
- Returns top-ranked personalized courses

This approach creates more intelligent recommendations compared to traditional keyword-based systems.

BACKEND IMPLEMENTATION

app.py

The `app.py` file acts as the main Flask application server and API controller.

APIs Implemented

Recommendation API

Endpoint: `/recommend`

Purpose:

Returns AI-based personalized course recommendations based on user interests and saved courses.

Trending Courses API

Endpoint: `/trending`

Purpose:

Returns top-rated trending courses.

Categories API

Endpoint: `/categories`

Purpose:

Returns all available course categories.

Courses by Category API

Endpoint: `/courses/<category>`

Purpose:

Returns courses filtered based on selected category.

All Courses API

Endpoint: `/courses/all`

Purpose:

Returns the complete course dataset.

RECOMMENDATION ENGINE IMPLEMENTATION

recommender.py

The `recommender.py` file contains the complete recommendation logic and machine learning workflow.

Features Implemented

Dataset Processing

- CSV dataset loading using Pandas
- Data preprocessing and cleaning
- Feature combination for recommendation logic

Embedding Generation

- Sentence Transformer model integration
- Course embedding generation
- User preference embedding generation

Similarity Matching

- Cosine similarity calculation
- Personalized recommendation ranking
- Category-based filtering

Trending Logic

- Sorting courses based on ratings
- Extraction of top-rated courses

Additional Functionalities

- Get all categories
- Get all courses
- Get category-specific courses

FRONTEND IMPLEMENTATION

The frontend was developed using React.js with modular reusable components and multiple interactive pages.

SYSTEM PAGES AND FEATURES

LOGIN PAGE

The Login page allows registered users to securely access the platform using their credentials.

Features

- Username and password validation
- Error handling for invalid login credentials
- LocalStorage-based login handling
- Automatic navigation after successful login
- Responsive login interface

SIGNUP PAGE

The Signup page is used for creating new user accounts.

Features

- New account registration
- Password confirmation validation
- Duplicate username checking
- LocalStorage-based user storage
- Automatic navigation after successful signup

PREFERENCES PAGE

The Preferences page allows users to select learning interests for personalized recommendations.

Features

- Multiple interest selection
- Dynamic technology domain buttons
- Personalized learning path setup
- User preference management

DASHBOARD PAGE

The Dashboard page displays AI-generated personalized recommendations.

Features

- AI-based course recommendations
- Search functionality
- Level-based filtering
- Rating-based sorting
- Recommendation statistics
- AI Match Score display
- Dynamic course rendering
- Saved course functionality

TRENDING PAGE

The Trending page displays highly rated and popular courses.

Features

- Trending course display
- Top-rated course listing
- Recommendation statistics section
- Dynamic course rendering

SAVED COURSES PAGE

The Saved Courses page stores bookmarked learning resources.

Features

- Saved course collection
- Remove saved course option
- Persistent LocalStorage handling
- Personalized learning library

COURSE LIBRARY PAGE

The Course Library page allows users to explore all available courses.

Features

- Complete course display
- Category-based filtering
- Dynamic category buttons
- Course exploration functionality

LEARN PAGE

The Learn page displays detailed course information.

Features

- Individual course details
- Course progress interface
- External course launching
- Suggested learning activities
- Course overview section

UI/UX IMPROVEMENTS

Several frontend enhancements were implemented to improve user experience and application appearance.

Improvements Included

- Modern green gradient theme
- Responsive layouts
- Interactive buttons and hover effects
- Dashboard statistics cards
- Hero section implementation
- Improved course card layouts
- Responsive grid structures
- Better typography and spacing
- Enhanced readability and navigation flow
- Consistent UI design across pages

FEATURES SUCCESSFULLY COMPLETED

The following modules and functionalities were successfully implemented:

- Login system
- Signup system
- Personalized recommendation dashboard
- AI recommendation engine
- Search functionality
- Filter and sorting functionality
- Saved course management
- Trending course recommendations
- Course library module
- Category filtering system
- Frontend-backend integration
- Responsive UI implementation
- Navigation and routing system
- Recommendation ranking logic
- Dynamic API communication
- Modular project structure

DATA STORAGE MECHANISM

LocalStorage Usage

The platform uses browser LocalStorage to temporarily store user-related data and maintain session information.

Stored Data

- Username
- Login status
- User interests
- Saved courses

Advantages

- Fast client-side access
- Maintains session after refresh
- Easy integration with React state management
- Simplified user data handling

CSV Dataset Usage

Course information is stored in:

`courses.csv`

Dataset Fields

- Course titles
- Categories
- Levels
- Ratings
- Tags
- URLs

TESTING PERFORMED

The following testing activities were successfully completed:

- API response testing
- Frontend-backend integration testing
- Recommendation accuracy testing
- Login and signup validation testing
- Search and filtering testing

- Navigation testing
- LocalStorage testing
- Responsive UI testing
- Saved course functionality testing

FINAL PROJECT OUTCOME

The Smart Learning Recommendation Platform was successfully developed as a functional AI-powered web application capable of delivering personalized learning experiences.

The project demonstrates:

- Full-stack web development
- Machine learning integration
- REST API communication
- Recommendation system implementation
- Responsive frontend development
- User personalization workflows
- Practical AI-based recommendation concepts

The application successfully fulfills the planned objectives and demonstrates practical implementation of intelligent recommendation systems in educational technology platforms.

CONCLUSION

The Smart Learning Recommendation Platform successfully demonstrates the implementation of an intelligent recommendation system integrated into a modern web application. The project combines AI-powered personalization with an interactive and responsive frontend to create a user-friendly learning environment.

The recommendation engine, powered by Sentence Transformers and cosine similarity, enables intelligent course recommendations based on user interests and learning preferences. The React frontend and Flask backend architecture provide efficient communication and modular system design.

Overall, the project demonstrates practical implementation of machine learning concepts, frontend-backend integration, and personalized recommendation workflows within an educational platform.